# Pandas

# Prise en main de Pandas

# Pandas pour la manipulation des données

- Pandas : des structures et fonctions puissantes
- Manipulation des données rapide, facile et expressive
- Basé sur NumPy avec des axes étiquetés pour éviter les erreurs d'alignement

# Les données tabulaires en data science

Les données tabulaires occupent une place essentielle en data science, d'où l'importance de Pandas

Les données tabulaires désignent des données structurées et organisées en lignes et en colonnes.

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| 0 | 1824 | Andrew Jackson | Democratic-Republican | 151271 | loss | 57.210122 |
| 1 | 1824 | John Quincy Adams | Democratic-Republican | 113142 | win | 42.789878 |
| 2 | 1828 | Andrew Jackson | Democratic | 642806 | win | 56.203927 |
| 3 | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| 4 | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |
| ... | ... | ... | ... | ... | ... | ... |
| 182 | 2024 | Donald Trump | Republican | 77303568 | win | 49.808629 |
| 183 | 2024 | Kamala Harris | Democratic | 75019230 | loss | 48.336772 |
| 184 | 2024 | Jill Stein | Green | 861155 | loss | 0.554864 |
| 185 | 2024 | Robert Kennedy | Independent | 756383 | loss | 0.487357 |
| 186 | 2024 | Chase Oliver | Libertarian Party | 650130 | loss | 0.418895 |

Chaque ligne correspond à une **observation** unique (e.g, une personne candidate à la présidence lors d'une année donnée).

Chaque colonne correspond à une **variable** ou un **attribut** décrivant l'observation ( e.g, le parti politique de la personne candidate)

# Series and DataFrames

- Principaux composants :
  - Series : structure unidimensionnelle avec index étiquetés
  - DataFrame : structure bidimensionnelle avec lignes et colonnes étiquetées

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| 0 | 1824 | Andrew Jackson | Democratic-Republican | 151271 | loss | 57.210122 |
| 1 | 1824 | John Quincy Adams | Democratic-Republican | 113142 | win | 42.789878 |
| 2 | 1828 | Andrew Jackson | Democratic | 642806 | win | 56.203927 |
| 3 | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| 4 | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |
| ... | ... | ... | ... | ... | ... | ... |
| 182 | 2024 | Donald Trump | Republican | 77303568 | win | 49.808629 |
| 183 | 2024 | Kamala Harris | Democratic | 75019230 | loss | 48.336772 |
| 184 | 2024 | Jill Stein | Green | 861155 | loss | 0.554864 |
| 185 | 2024 | Robert Kennedy | Independent | 756383 | loss | 0.487357 |
| 186 | 2024 | Chase Oliver | Libertarian Party | 650130 | loss | 0.418895 |

A `DataFrame`

```
0          Andrew Jackson
1        John Quincy Adams
2          Andrew Jackson
3        John Quincy Adams
4          Andrew Jackson
              ...
182          Donald Trump
183          Kamala Harris
184            Jill Stein
185        Robert Kennedy
186          Chase Oliver
Name: Candidate, Length: 187, dtype: object
```

A `Series` named "Candidate"

```
from pandas import Series, DataFrame
import pandas as pd
```

# Series

- Tableau unidimensionnel contenant des **valeurs du même type**
- Utilise **une séquence d'étiquettes, appelée index**, pour identifier chaque valeur

```
import pandas as pd
s = pd.Series(["welcome", "to", "data 100"])
```

```
0    welcome
1         to
2   data 100
dtype: object
```

**Index**, accessible via `s.index`

```
RangeIndex(start=0, stop=3, step=1)
```

**Values**, accessible via `s.values`

```
array(['welcome', 'to', 'data 100'], dtype=object)
```

# Construire les `Series`

- Chaque élément d'une Series peut être identifié par une **étiquette (label) d'index**
- Ces étiquettes sont **spécifiées via une liste d'index** lors de l'initialisation de la Series

```
s = pd.Series([-1, 10, 2], index = ["a", "b", "c"])
```

```
a    -1
b    10
c     2
dtype: int64
```

```
s.index
```

```
Index(['a', 'b', 'c'], dtype='object')
```

- L'**index d'une Series peut également être modifié** après sa création.

```
s.index = ["first", "second", "third"]
```

```
first     -1
second    10
third      2
dtype: int64
```

```
s.index
```

```
Index(['first', 'second', 'third'], dtype='object')
```

# La selection dans les `Series`

- On peut sélectionner une **valeur unique** ou un **ensemble de valeurs** en utilisant :
  - Une étiquette unique
  - Une liste d'étiquettes
  - Une condition de filtrage

```
s = pd.Series([4, -2, 0, 6], index = ["a", "b", "c", "d"])
```

```
a    4
b   -2
c    0
d    6
dtype: int64
```

# La selection dans `Series`

- On peut sélectionner une **valeur unique** ou un **ensemble de valeurs** en utilisant :
  - **Une étiquette unique**
  - Une liste d'étiquettes
  - Une condition de filtrage

```
s = pd.Series([4, -2, 0, 6], index = ["a", "b", "c", "d"])
```

```
a    4
b   -2
c    0
d    6
dtype: int64
```

```
s["a"]
```

```
4
```

A single string label

A single value

# La selection dans `Series`

- On peut sélectionner une **valeur unique** ou un **ensemble de valeurs** en utilisant :
  - Une étiquette unique
  - **Une liste d'étiquettes**
  - Une condition de filtrage

```
s = pd.Series([4, -2, 0, 6], index = ["a", "b", "c", "d"])
```

```
a    4
b   -2
c    0
d    6
dtype: int64
```

```
s[["a", "c"]]
```

```
a    4
c    0
dtype: int64
```

Une liste d'étiquettes (label) (chaînes de caractères)

La `Series`

# La selection dans `Series`

- On peut sélectionner une **valeur unique** ou un **ensemble de valeurs** en utilisant :
  - Une étiquette unique
  - Une liste d'étiquettes
  - **Une condition de filtrage**

```
s = pd.Series([4, -2, 0, 6], index = ["a", "b", "c", "d"])
```

```
a    4
b   -2
c    0
d    6
dtype: int64
```

- Étapes
  - Création d'un masque booléen via une condition
  - Application du masque pour filtrer la Series
  - Résultat : sous-ensemble des valeurs vérifiant la condition

```
s > 0
```

```
a     True
b    False
c    False
d     True
dtype: bool
```

```
s[s > 0]
```

Boolean mask

```
a    4
d    6
dtype: int64
```

# DataFrames of Series!

- On considère principalement les **Series** comme des **colonnes d'un DataFrame**.

- Un **DataFrame** peut être vu comme une **collection de Series partageant le même index**.

```
0       1824
1       1824
2       1828
3       1828
4       1832
       ...
182     2024
183     2024
184     2024
185     2024
186     2024
Name: Year,
```

The Series `"Year"`

```
0          Andrew Jackson
1      John Quincy Adams
2          Andrew Jackson
3      John Quincy Adams
4          Andrew Jackson
                 ...
182          Donald Trump
183         Kamala Harris
184             Jill Stein
185        Robert Kennedy
186          Chase Oliver
Name: Candidate,
```

The Series `"Candidate"`

[...]

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| **0** | 1824 | Andrew Jackson | Democratic-Republican | 151271 | loss | 57.210122 |
| **1** | 1824 | John Quincy Adams | Democratic-Republican | 113142 | win | 42.789878 |
| **2** | 1828 | Andrew Jackson | Democratic | 642806 | win | 56.203927 |
| **3** | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| **4** | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |
| **...** | ... | ... | ... | ... | ... | ... |
| **182** | 2024 | Donald Trump | Republican | 77303568 | win | 49.808629 |
| **183** | 2024 | Kamala Harris | Democratic | 75019230 | loss | 48.336772 |
| **184** | 2024 | Jill Stein | Green | 861155 | loss | 0.554864 |
| **185** | 2024 | Robert Kennedy | Independent | 756383 | loss | 0.487357 |
| **186** | 2024 | Chase Oliver | Libertarian Party | 650130 | loss | 0.418895 |

The DataFrame `elections`

# Pandas, Part I

Introduction à `pandas:` syntaxe, operateurs, et fonctions

# Recap: DataFrames & Series

- Un **DataFrame** est une structure bidimensionnelle composée de plusieurs **Series alignées sur un index commun**.
- Une **Series** est constituée d'un **index** (étiquettes) et d'un **ensemble de valeurs** stockées sous forme de tableau

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| 0 | 1824 | Andrew Jackson | Democratic-Republican | 151271 | loss | 57.210122 |
| 1 | 1824 | John Quincy Adams | Democratic-Republican | 113142 | win | 42.789878 |
| 2 | 1828 | Andrew Jackson | Democratic | 642806 | win | 56.203927 |
| 3 | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| 4 | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |
| ... | ... | ... | ... | ... | ... | ... |
| 182 | 2024 | Donald Trump | Republican | 77303568 | win | 49.808629 |
| 183 | 2024 | Kamala Harris | Democratic | 75019230 | loss | 48.336772 |
| 184 | 2024 | Jill Stein | Green | 861155 | loss | 0.554864 |
| 185 | 2024 | Robert Kennedy | Independent | 756383 | loss | 0.487357 |
| 186 | 2024 | Chase Oliver | Libertarian Party | 650130 | loss | 0.418895 |

**A `DataFrame`**

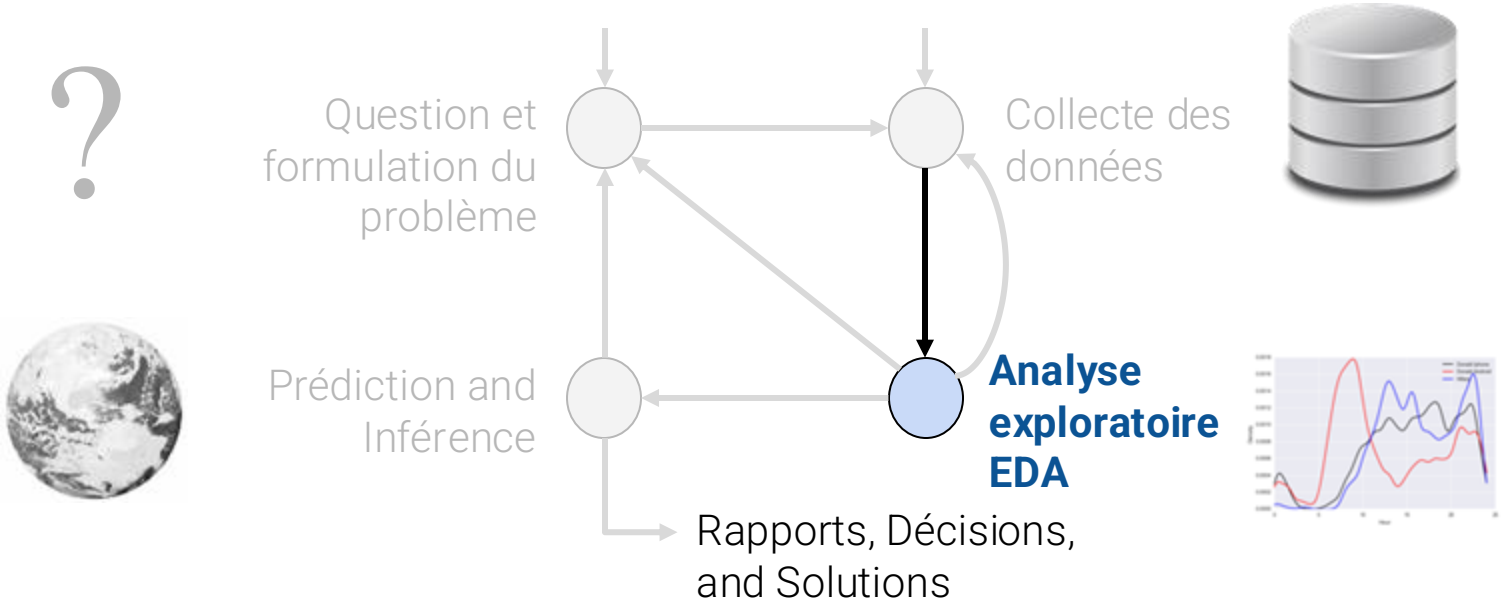
**Étiquettes (index) associées à chaque observation**

```
0          Andrew Jackson
1        John Quincy Adams
2          Andrew Jackson
3        John Quincy Adams
4          Andrew Jackson
             ...
182         Donald Trump
183         Kamala Harris
184           Jill Stein
185        Robert Kennedy
186         Chase Oliver
Name: Candidate, Length: 187, dtype: object
```

**Les valeurs de la Series**

**Le nom de la `Series` : "Candidate"**

# Rappel du Cycle de vie en Data Science




Explorer et nettoyer des données tabulaires
De la démarche en data science à l'implémentation avec Pandas

Data Science en Pratique
EDA, Nettoyage de données, traitement de texte (regular expressions)

# DataFrames, Series, et Index

- **DataFrames, Series, et Index**
- Extraction des données avec `loc`, `iloc`, and `[]`

https://github.com/pandas-dev/pandas/blob/main/pandas/core/frame.py

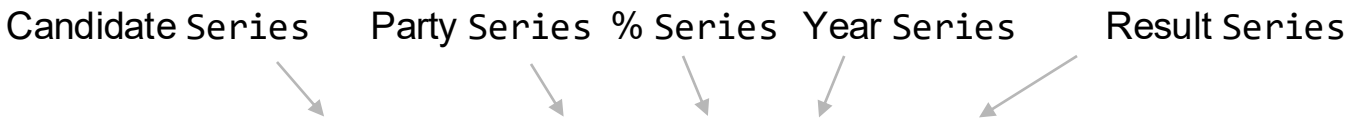
```
data : ndarray (structured or homogeneous), Iterable, dict, or DataFrame
    Dict can contain Series, arrays, constants, dataclass or list-like objects. If
    data is a dict, column order follows insertion-order. If a dict contains Series
    which have an index defined, it is aligned by its index. This alignment also
    occurs if data is a Series or a DataFrame itself. Alignment is done on
    Series/DataFrame inputs.

    If data is a list of dicts, column order follows insertion-order.

index : Index or array-like
    Index to use for resulting frame. Will default to RangeIndex if
    no indexing information part of input data and no index provided.
columns : Index or array-like
    Column labels to use for resulting frame when data does not have them,
    defaulting to RangeIndex(0, 1, 2, ..., n). If data contains column labels,
    will perform column selection instead.
dtype : dtype, default None
    Data type to force. Only a single dtype is allowed. If None, infer.
    If ``data`` is DataFrame then is ignored. Missing values in ``data``
    are not cast: for example, with ``dtype=str`` a ``NaN`` entry stays
    missing rather than becoming the string ``"nan"``.
copy : bool or None, default None
```

# Relation entre DataFrames, Series et index

- On peut considérer un **DataFrame** comme une collection de **Series** partageant toutes le même index.
- Les Series *Candidate*, *Party*, *%*, *Year* et *Result* partagent toutes un index allant de 0 à 5.




| | Candidate | Party | % | Year | Result |
|---|---|---|---|---|---|
| 0 | Obama | Democratic | 52.9 | 2008 | win |
| 1 | McCain | Republican | 45.7 | 2008 | loss |
| 2 | Obama | Democratic | 51.1 | 2012 | win |
| 3 | Romney | Republican | 47.2 | 2012 | loss |
| 4 | Clinton | Democratic | 48.2 | 2016 | loss |
| 5 | Trump | Republican | 46.1 | 2016 | win |

# Création d'un `DataFrame`

Il existe de nombreuses façons de créer un DataFrame ! Parmi les plus courantes :

- À partir d'un fichier CSV.
- À partir d'un dictionnaire de colonnes (chaque colonne pouvant être une liste, un tableau (*array*) ou une Series).
- À partir d'une liste de lignes (chaque ligne pouvant être un dictionnaire, une liste, un tableau ou une Series
    - ou à partir d'un tableau bidimensionnel (2D).

- Comprendre la syntaxe que nous vous présentons est bien plus important que de la mémoriser.
- Savoir **ce que l'on veut faire** est plus essentiel que savoir précisément **comment l'implémenter en code**.
    - ✓ Google, la documentation, et les LLMs* sont utiles pour répondre au « comment ».

# Création d'un `DataFrame` à partir…

## …d'un fichier CSV:

```
elections = pd.read_csv("data/elections.csv", ...)
```

## …dictionnaire de colonnes :

```
pd.DataFrame(
    {"Fruit": ["Kiwi",
"Orange"],
     "Price": [5.49, 3.99]}
)
```

## …Series:

```
pd.DataFrame(s_a)
```

```
s_a.to_frame()
```

## …d'une liste de lignes :

```
pd.DataFrame(
    [{"Fruit": "Kiwi", "Price": 5.49},
     {"Fruit": "Orange", "Price": 3.99}]
)
```

```
pd.DataFrame(
    [["Kiwi", 5.49],
     ["Orange", 3.99]],
    columns=["Fruit", "Price"]
)
```

# Explorer la structure d'un DataFrame : `Index, Columns, & shape`

```
elections = pd.read_csv("data/elections.csv", index_col = "Candidate")
```

`DataFrame.index`: pour extraire les labels des lignes.



```
elections.index
```

```
Index(['Democratic-Republican', 'Democratic-Republican', 'Democratic',
       'National Republican', 'Democratic', 'National Republican',
       'Anti-Masonic', 'Whig', 'Democratic', 'Whig',
       ...
       'Green', 'Democratic', 'Republican', 'Libertarian', 'Green',
       'Republican', 'Democratic', 'Green', 'Independent',
       'Libertarian Party'],
      dtype='object', name='Party', length=187)
```

`DataFrame.columns`: pour extraire les labels des colonnes.

```
elections.columns
```

```
Index(['Candidate', 'Year', 'Popular vote', 'Result', '%'], dtype='object')
```

`DataFrame.shape`: pour les dimensions.



```
elections.shape
```

```
(187, 6)
```

# Extraction des données avec `loc`, `iloc`, and `[]`

- DataFrames, Series, et Index
- **Extraction des données avec `loc`, `iloc`, and `[]`**

# Extraction des Données

Il existe de nombreuses façons d'extraire des lignes et des colonnes d'intérêt à partir d'un DataFrame :

- Récupérer les n premières ou dernières lignes du DataFrame.
- Extraire des données à partir d'une étiquette (label) spécifique.
- Extraire des données à une position donnée.

Nous verrons que ces trois méthodes sont utiles dans des contextes différents.

# .head &.tail

Les cas les plus simples : extraire les n premières ou les n dernières lignes.

- df.head(n) renvoie les n premières lignes du DataFrame df.
- df.tail(n) renvoie les n dernières lignes.

`elections`

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| 0 | 1824 | Andrew Jackson | Democratic-Republican | 151271 | loss | 57.210122 |
| 1 | 1824 | John Quincy Adams | Democratic-Republican | 113142 | win | 42.789878 |
| 2 | 1828 | Andrew Jackson | Democratic | 642806 | win | 56.203927 |
| 3 | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| 4 | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |
| ... | ... | ... | ... | ... | ... | ... |
| 182 | 2024 | Donald Trump | Republican | 77303568 | win | 49.808629 |
| 183 | 2024 | Kamala Harris | Democratic | 75019230 | loss | 48.336772 |
| 184 | 2024 | Jill Stein | Green | 861155 | loss | 0.554864 |
| 185 | 2024 | Robert Kennedy | Independent | 756383 | loss | 0.487357 |
| 186 | 2024 | Chase Oliver | Libertarian Party | 650130 | loss | 0.418895 |

`elections.head(5)`

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| 0 | 1824 | Andrew Jackson | Democratic-Republican | 151271 | loss | 57.210122 |
| 1 | 1824 | John Quincy Adams | Democratic-Republican | 113142 | win | 42.789878 |
| 2 | 1828 | Andrew Jackson | Democratic | 642806 | win | 56.203927 |
| 3 | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| 4 | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |

`elections.tail(5)`

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| 182 | 2024 | Donald Trump | Republican | 77303568 | win | 49.808629 |
| 183 | 2024 | Kamala Harris | Democratic | 75019230 | loss | 48.336772 |
| 184 | 2024 | Jill Stein | Green | 861155 | loss | 0.554864 |
| 185 | 2024 | Robert Kennedy | Independent | 756383 | loss | 0.487357 |
| 186 | 2024 | Chase Oliver | Libertarian Party | 650130 | loss | 0.418895 |

Remarque : si vous ne spécifiez pas n, `elections.head()` renvoie par défaut 5 lignes.

# Extraction basée sur les labels : .loc

# Extraction basée sur les labels : .loc

- Une tâche plus complexe : extraire des données en fonction de labels spécifiques de lignes ou de colonnes.



`df.loc[ligne_labels, colonne_labels]`

`.loc` *stands for location*

- L'accesseur .loc permet de spécifier les étiquettes des lignes et des colonnes que l'on souhaite extraire.

Labels des lignes (index)

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| 0 | 1824 | Andrew Jackson | Democratic-Republican | 151271 | loss | 57.210122 |
| 1 | 1824 | John Quincy Adams | Democratic-Republican | 113142 | win | 42.789878 |
| 2 | 1828 | Andrew Jackson | Democratic | 642806 | win | 56.203927 |
| 3 | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| 4 | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |
| ... | ... | ... | ... | ... | ... | ... |
| 182 | 2024 | Donald Trump | Republican | 77303568 | win | 49.808629 |
| 183 | 2024 | Kamala Harris | Democratic | 75019230 | loss | 48.336772 |
| 184 | 2024 | Jill Stein | Green | 861155 | loss | 0.554864 |
| 185 | 2024 | Robert Kennedy | Independent | 756383 | loss | 0.487357 |
| 186 | 2024 | Chase Oliver | Libertarian Party | 650130 | loss | 0.418895 |

Labels des colonnes

# Extraction basée sur les labels : .loc

Chacun des deux* arguments de .loc peut être :

- une valeur unique
- une liste
- un découpage (*slice*) avec borne de droite incluse
- un masque booléen

**Si un seul argument est utilisé, pandas l'applique aux lignes et renvoie toutes les colonnes.*

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| **0** | 1824 | Andrew Jackson | Democratic-Republican | 151271 | loss | 57.210122 |
| **1** | 1824 | John Quincy Adams | Democratic-Republican | 113142 | win | 42.789878 |
| **2** | 1828 | Andrew Jackson | Democratic | 642806 | win | 56.203927 |
| **3** | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| **4** | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |
| **...** | ... | ... | ... | ... | ... | ... |
| **182** | 2024 | Donald Trump | Republican | 77303568 | win | 49.808629 |
| **183** | 2024 | Kamala Harris | Democratic | 75019230 | loss | 48.336772 |
| **184** | 2024 | Jill Stein | Green | 861155 | loss | 0.554864 |
| **185** | 2024 | Robert Kennedy | Independent | 756383 | loss | 0.487357 |
| **186** | 2024 | Chase Oliver | Libertarian Party | 650130 | loss | 0.418895 |

# Extraction basée sur les labels : .loc

Chacun des deux* arguments de .loc peut être :

- Une valeur unique
- une liste
- un découpage (*slice*) avec borne de droite incluse
- un masque booléen

*Valeur Unique* *Valeur Unique*

```
elections.loc[0, "Candidate"]
```

```
'Andrew Jackson'
```

Nous avons extrait l'élément unique correspondant au label de ligne **0** et au label de colonne **« Candidate »**.

*Liste* *Valeur Unique*

```
elections.loc[[87, 25, 179], "Popular vote"]
```

```
87     15761254
25       848019
179    74216154
Name: Popular vote, dtype: int64
```

Nous avons extrait un sous-ensemble de la colonne **« Popular vote »** sous forme de *Series*.

# Extraction basée sur les labels : .loc

Chacun des deux* arguments de .loc peut être :

- une valeur unique
- une liste
- un découpage (*slice*) avec borne de droite incluse
- un masque booléen

*Liste* *Liste*

```
elections.loc[[87, 25, 179], ["Year", "Candidate", "Result"]]
```

Sélectionner les lignes avec les labels 87, 25, and 179.

| | Year | Candidate | Result |
|---|---|---|---|
| **87** | 1932 | Herbert Hoover | loss |
| **25** | 1860 | John C. Breckinridge | loss |
| **179** | 2020 | Donald Trump | loss |

Sélectionner les colonnes portant les labels "Year", "Candidate" et "Result"

# Extraction basée sur les labels : .loc

Chacun des deux* arguments de .loc peut être :

- une valeur unique
- une liste
- un découpage (*slice*) avec borne de droite incluse
- un masque booléen

*Liste* *Slice*

```
elections.loc[[87, 25, 179], "Popular vote":"%"]
```

Sélectionner toutes les colonnes allant de "Popular vote" jusqu'à "%", inclus.

Sélectionner les lignes avec les labels 87, 25, and 179.

| | Popular vote | Result | % |
|---|---|---|---|
| **87** | 15761254 | loss | 39.830594 |
| **25** | 848019 | loss | 18.138998 |
| **179** | 74216154 | loss | 46.858542 |

# Extraction basée sur les labels : .loc

- Pour extraire toutes les colonnes ou toutes les lignes, on utilise les deux-points (:)

```
elections.loc[:, ["Year", "Candidate", "Result"]]
```

Toutes les lignes avec les labels "Year", "Candidate", et "Result".

Les points de suspension ( … ) indiquent que d'autres lignes ne sont pas affichées.

| | Year | Candidate | Result |
|---|---|---|---|
| 0 | 1824 | Andrew Jackson | loss |
| 1 | 1824 | John Quincy Adams | win |
| 2 | 1828 | Andrew Jackson | win |
| 3 | 1828 | John Quincy Adams | loss |
| 4 | 1832 | Andrew Jackson | win |
| … | … | … | … |
| 182 | 2024 | Donald Trump | win |
| 183 | 2024 | Kamala Harris | loss |
| 184 | 2024 | Jill Stein | loss |
| 185 | 2024 | Robert Kennedy | loss |
| 186 | 2024 | Chase Oliver | loss |

```
elections.loc[[87, 25, 179], :]
```

**Toutes les colonnes pour les lignes portant les labels** 87, 25, 179.

| | Candidate | Year | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| 87 | Herbert Hoover | 1932 | Republican | 15761254 | loss | 39.830594 |
| 25 | John C. Breckinridge | 1860 | Southern Democratic | 848019 | loss | 18.138998 |
| 179 | Donald Trump | 2020 | Republican | 74216154 | loss | 46.858542 |

Technical note: `elections.loc[[87, 25, 179]]` is the same as `elections.loc[[87, 25, 179], :]`.
With one argument, `.loc` assumes the second argument is a colon (:).

# Extraction basée sur les labels : .loc

Chacun des deux* arguments de .loc peut être :

- une valeur unique
- une liste
- un découpage (*slice*) avec borne de droite incluse
- un masque booléen

*Masque* *Liste*

```
elections.loc[elections["Year"] == 2008, ["Year", "Candidate"]]
```

Série de longueur 187 dont chaque élément est soit True soit False, où True apparaît pour chaque ligne vérifiant "Year" == 2008

| | Year | Candidate |
|---|---|---|
| **162** | 2008 | Barack Obama |
| **163** | 2008 | Bob Barr |
| **164** | 2008 | Chuck Baldwin |
| **165** | 2008 | Cynthia McKinney |
| **166** | 2008 | John McCain |
| **167** | 2008 | Ralph Nader |

# Extraction basée sur les indices: `.iloc`

# Extraction basée sur les indices: `.iloc`

Sélectionner des données en utilisant des positions entières (indices numériques), et non des labels.

`.iloc` stands for integer location



```
df.iloc[ligne_entier, col_entier]
```

Indices entiers des lignes

Indices entiers des colonnes

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| | Year | Candidate | Party | Popular vote | Result | % |
| 0 | 1824 | Andrew Jackson | Democratic-Republican | 151271 | loss | 57.210122 |
| 1 | 1824 | John Quincy Adams | Democratic-Republican | 113142 | win | 42.789878 |
| 2 | 1828 | Andrew Jackson | Democratic | 642806 | win | 56.203927 |
| 3 | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| 4 | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |
| ... | ... | ... | ... | ... | ... | ... |

# Extraction basée sur les indices: `.iloc`

Chacun des deux* arguments de .iloc peut être :

- une valeur entière unique
- une liste
- un découpage (*slice*) (exclusif sur la borne de droite, comme range(a, b))

*Si un seul argument est utilisé, pandas l'applique aux lignes et renvoie toutes les colonnes.*

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| **0** | 1824 | Andrew Jackson | Democratic-Republican | 151271 | loss | 57.210122 |
| **1** | 1824 | John Quincy Adams | Democratic-Republican | 113142 | win | 42.789878 |
| **2** | 1828 | Andrew Jackson | Democratic | 642806 | win | 56.203927 |
| **3** | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| **4** | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |
| **...** | ... | ... | ... | ... | ... | ... |
| **182** | 2024 | Donald Trump | Republican | 77303568 | win | 49.808629 |
| **183** | 2024 | Kamala Harris | Democratic | 75019230 | loss | 48.336772 |
| **184** | 2024 | Jill Stein | Green | 861155 | loss | 0.554864 |
| **185** | 2024 | Robert Kennedy | Independent | 756383 | loss | 0.487357 |
| **186** | 2024 | Chase Oliver | Libertarian Party | 650130 | loss | 0.418895 |

# Extraction basée sur les indices: `.iloc`

Chacun des deux* arguments de .iloc peut être :

- une valeur entière unique
- une liste
- un découpage (*slice*) (exclusif sur la borne de droite, comme range(a, b))

```
elections.iloc[0, 1]
```

```
'Andrew Jackson'
```

Extraire l'élément unique situé à la position de ligne 0 et à la position de colonne 1.

```
elections.iloc[[1, 2, 3], 1]
```

```
1    John Quincy Adams
2       Andrew Jackson
3    John Quincy Adams
Name: Candidate, dtype: object
```

Comme avec .loc, le résultat d'un argument correspondant à une seule valeur est une Series.

Extraire les lignes d'indices 1, 2 et 3 dans la colonne de position 1.

# Extraction basée sur les indices: `.iloc`

Chacun des deux* arguments de .iloc peut être :

- une valeur entière unique
- une liste
- un découpage (*slice*) (exclusif sur la borne de droite, comme range(a, b))

```
elections.iloc[[1, 2, 3], [0, 1, 2]]
```

Sélectionner les lignes de position 1, 2, and 3.

| | Year | Candidate | Party |
|---|---|---|---|
| **1** | 1824 | John Quincy Adams | Democratic-Republican |
| **2** | 1828 | Andrew Jackson | Democratic |
| **3** | 1828 | John Quincy Adams | National Republican |

Sélectionner les colonnes au positions 0, 1, and 2.

# Extraction basée sur les indices: `.iloc`

Chacun des deux* arguments de .iloc peut être :

- une valeur entière unique
- une liste
- un découpage (*slice*) (exclusif sur la borne de droite, comme range(a, b))

```
elections.iloc[[1, 2, 3], 0:3]
```

| | 0<br>Year | 1<br>Candidate | 2<br>Party |
|---|---|---|---|
| **1** | 1824 | John Quincy Adams | Democratic-Republican |
| **2** | 1828 | Andrew Jackson | Democratic |
| **3** | 1828 | John Quincy Adams | National Republican |

Sélectionner les lignes aux positions1, 2, **and 3**.

Sélectionner toutes les colonnes de l'indice entier 0 à l'indice entier 2.

☝ **Le découpage (*slicing*) est exclusif sur la borne de droite** !

# Extraction basée sur les indices: `.iloc`

Comme avec `.loc`, on peut utiliser le symbole « : » avec `.iloc` pour extraire toutes les lignes ou toutes les colonnes.

```
elections.iloc[:, 0:3]
```

| | Year | Candidate | Result |
|---|---|---|---|
| **0** | 1824 | Andrew Jackson | loss |
| **1** | 1824 | John Quincy Adams | win |
| **2** | 1828 | Andrew Jackson | win |
| **3** | 1828 | John Quincy Adams | loss |
| **4** | 1832 | Andrew Jackson | win |
| **...** | ... | ... | ... |
| **182** | 2024 | Donald Trump | win |
| **183** | 2024 | Kamala Harris | loss |
| **184** | 2024 | Jill Stein | loss |
| **185** | 2024 | Robert Kennedy | loss |
| **186** | 2024 | Chase Oliver | loss |

Sélectionner toutes les lignes des colonnes aux positions entières 0 à 2.

# Récap

À retenir :

`.loc` effectue une extraction basée sur `les labels`
`.iloc` effectue une extraction basée sur `les indices entiers`

Lorsqu'il s'agit de choisir entre `.loc` et **`.iloc`**, on privilégie généralement **`.loc`**.

- Plus sûr : si l'ordre des données est modifié dans une base de données publique, le code reste valide.
- Plus lisible : il est plus facile de comprendre ce que signifie `elections.loc[:, ["Year", "Candidate", "Result"]]` que `elections.iloc[:, [0, 1, 4]]`

**`.iloc`** reste néanmoins utile.

Exemple : pour un DataFrame de films triés par recettes, on peut utiliser **`.iloc`** pour obtenir la valeur médiane (c'est-à-dire accéder à l'élément central).

# Extraction dépendante du contexte: [ ]

## Extraction dépendante du contexte: [ ]

L'indexation avec [ ] ne prend qu'un seul argument, qui peut servir à sélectionner des lignes ou des colonnes :

- Si vous utilisez un label unique ou une liste de labels, pandas l'utilise pour extraire une ou plusieurs colonnes.
- Si vous utilisez un découpage (slice) ou un masque booléen, pandas l'utilise pour extraire des lignes.

Cela signifie que [ ] est sensible au contexte : selon l'argument fourni, son comportement change.

Cette syntaxe est populaire, car elle permet de saisir rapidement les opérations courantes

….Cependant, elle peut être difficile à lire et ne fonctionne pas toujours pour des opérations avancées.

# Extraction dépendante du contexte: [ ]

[ ] ne prend qu'un seul argument, qui peut être :

- un label de colonne unique
- une liste de labels de colonnes
- un découpage (slice) d'indices entiers de lignes
- un masque booléen appliqué aux lignes



```
elections["Candidate"]
0          Andrew Jackson
1       John Quincy Adams
2          Andrew Jackson
3       John Quincy Adams
4          Andrew Jackson
              ...
182          Donald Trump
183         Kamala Harris
184            Jill Stein
185        Robert Kennedy
186          Chase Oliver
Name: Candidate, Length: 187, dtype: object
```

*Sélectionne la colonne "Candidate" comme une* `Series`.

`elections[["Candidate"]]` renvoie un DataFrame à une seule colonne.

# Extraction dépendante du contexte: [ ]

[ ] ne prend qu'un seul argument, qui peut être :

- un label de colonne unique
- une liste de labels de colonnes
- un découpage (slice) d'indices entiers de lignes
- un masque booléen appliqué aux lignes

*Attention à ne pas oublier les crochets interne*

```
elections[["Year", "Candidate", "Result"]]
```



| | Year | Candidate | Result |
|---|---|---|---|
| 0 | 1824 | Andrew Jackson | loss |
| 1 | 1824 | John Quincy Adams | win |
| 2 | 1828 | Andrew Jackson | win |
| 3 | 1828 | John Quincy Adams | loss |
| 4 | 1832 | Andrew Jackson | win |
| ... | ... | ... | ... |
| 182 | 2024 | Donald Trump | win |
| 183 | 2024 | Kamala Harris | loss |
| 184 | 2024 | Jill Stein | loss |
| 185 | 2024 | Robert Kennedy | loss |
| 186 | 2024 | Chase Oliver | loss |

# Extraction dépendante du contexte: [ ]

[ ] ne prend qu'un seul argument, qui peut être :

- un label de colonne unique
- une liste de labels de colonnes
- un découpage (slice) d'indices entiers de lignes
- un masque booléen appliqué aux lignes

L'utilisation d'un découpage (slice) avec l'indexation [] est toujours :

- exclu sur la borne de droite
- interprété comme des positions entières (style .iloc)
  (même si les labels sont des entiers !)

```
elections[3:7]
```

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| **3** | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| **4** | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |
| **5** | 1832 | Henry Clay | National Republican | 484205 | loss | 37.603628 |
| **6** | 1832 | William Wirt | Anti-Masonic | 100715 | loss | 7.821583 |

# Extraction dépendante du contexte: [ ]

[ ] ne prend qu'un seul argument, qui peut être :

- un label de colonne unique
- une liste de labels de colonnes
- un découpage (slice) d'indices entiers de lignes
- un masque booléen appliqué aux lignes



```
elections[elections["Year"] == 2008]
```

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| **162** | 2008 | Barack Obama | Democratic | 69498516 | win | 53.023510 |
| **163** | 2008 | Bob Barr | Libertarian | 523715 | loss | 0.399565 |
| **164** | 2008 | Chuck Baldwin | Constitution | 199750 | loss | 0.152398 |
| **165** | 2008 | Cynthia McKinney | Green | 161797 | loss | 0.123442 |
| **166** | 2008 | John McCain | Republican | 59948323 | loss | 45.737243 |
| **167** | 2008 | Ralph Nader | Independent | 739034 | loss | 0.563842 |

## Construction de masques booléens complexes avec des opérateurs logiques

Les `Series` booléennes peuvent être combinées élément par élément à l'aide d'opérateurs booléens.

- L'opérateur `&` applique un **AND** entre `boolean_series_1` et `boolean_series_2`.
- L'opérateur `|` applique un **OR** entre `boolean_series_1` et `boolean_series_2`.

```
elections[(elections["Year"] == 2008) | (elections["%"] > 60)]
```

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| **79** | 1920 | Warren Harding | Republican | 16144093 | win | 60.574501 |
| **91** | 1936 | Franklin Roosevelt | Democratic | 27752648 | win | 60.978107 |
| **114** | 1964 | Lyndon Johnson | Democratic | 43127041 | win | 61.344703 |
| **120** | 1972 | Richard Nixon | Republican | 47168710 | win | 60.907806 |
| **162** | 2008 | Barack Obama | Democratic | 69498516 | win | 53.023510 |
| **163** | 2008 | Bob Barr | Libertarian | 523715 | loss | 0.399565 |
| **164** | 2008 | Chuck Baldwin | Constitution | 199750 | loss | 0.152398 |
| **165** | 2008 | Cynthia McKinney | Green | 161797 | loss | 0.123442 |
| **166** | 2008 | John McCain | Republican | 59948323 | loss | 45.737243 |
| **167** | 2008 | Ralph Nader | Independent | 739034 | loss | 0.563842 |

Lignes correspondant à l'année 2008 OU à un candidat ayant remporté plus de 60 % des voix (ou les deux !).

# Opérateurs bit à bit (*Bitwise Operators*)

- `&` et `|` sont des exemples d'opérateurs au niveau des bits (*bitwise operators*). Ils permettent d'appliquer plusieurs conditions logiques.
- Si `p` et `q` sont des tableaux booléens ou des `Series` :

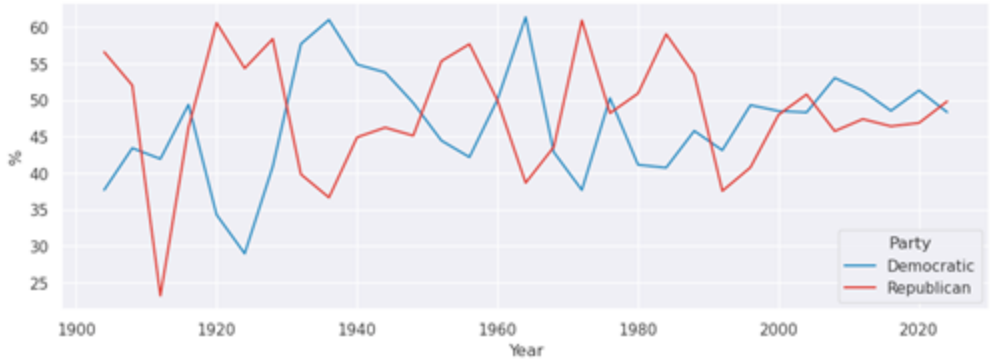
| Symbole | Utilisation | Signification |
|---|---|---|
| `~` | `~p` | NOT `p` |
| `\|` | `p \| q` | `p` OR `q` |
| `&` | `p & q` | `p` AND `q` |
| `^` | `p ^ q` | `p` XOR `q` |

Assurez-vous **toujours** d'utiliser des parenthèses `( )` autour de chaque condition lors de l'utilisation de `&` et `|` (ainsi que `^`).



p XOR q : soit p, soit q, mais pas les deux. Ou exclusif !

# Exemple: sélection des données pour notre graphique

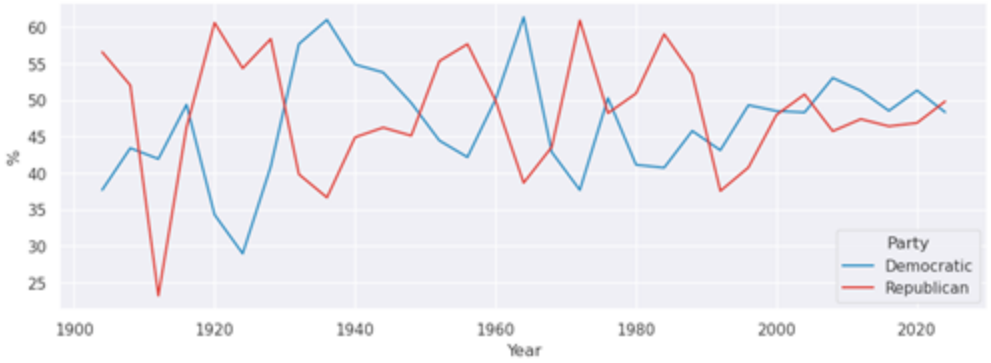



After 1900 only

Dem/Rep only

| | Year | Party | % |
|---|---|---|---|
| 57 | 1904 | Democratic | 37.685116 |
| 60 | 1904 | Republican | 56.562787 |
| 64 | 1908 | Democratic | 43.414640 |
| 65 | 1908 | Republican | 52.013300 |
| 69 | 1912 | Republican | 23.218466 |
| ... | ... | ... | ... |

# Exemple: sélection des données pour notre graphique



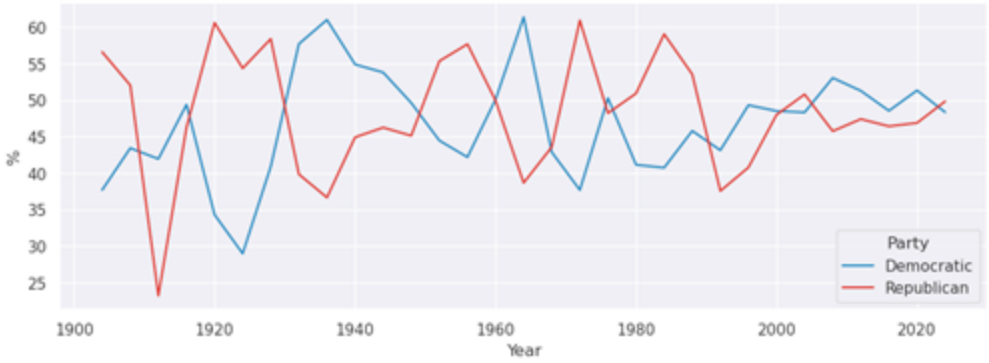

After 1900 only

Dem/Rep only

| | Year | Party | % |
|---|---|---|---|
| 57 | 1904 | Democratic | 37.685116 |
| 60 | 1904 | Republican | 56.562787 |
| 64 | 1908 | Democratic | 43.414640 |
| 65 | 1908 | Republican | 52.013300 |
| 69 | 1912 | Republican | 23.218466 |
| ... | ... | ... | ... |

```
elections.loc[
    (
        ((elections['Party'] == 'Democratic') | (elections['Party'] == 'Republican')) &
        (elections['Year'] > 1900)
    ),
    ['Year', 'Party', '%']
]
```

# Elections Dataset: What Happened in 1992?







```
elections[elections['Year'] == 1992]
```

| Candidate | Year | Party | Popular vote | Result | % |
|---|---|---|---|---|---|
| **Andre Marrou** | 1992 | Libertarian | 290087 | loss | 0.278516 |
| **Bill Clinton** | 1992 | Democratic | 44909806 | win | 43.118485 |
| **Bo Gritz** | 1992 | Populist | 106152 | loss | 0.101918 |
| **George H. W. Bush** | 1992 | Republican | 39104550 | loss | 37.544784 |
| **Ross Perot** | 1992 | Independent | 19743821 | loss | 18.956298 |

# Les indices ne correspondent pas toujours aux numéros de ligne

L'indice (*index*) (c'est-à-dire les labels de lignes) peut également :

- être non numérique
- porter un nom, par exemple « Candidate »

```
# Creating a DataFrame from a CSV file and specifying the Index column
elections = pd.read_csv("data/elections.csv", index_col = "Candidate")
```

| Candidate | Year | Party | Popular vote | Result | % |
|---|---|---|---|---|---|
| **Andrew Jackson** | 1824 | Democratic-Republican | 151271 | loss | 57.210122 |
| **John Quincy Adams** | 1824 | Democratic-Republican | 113142 | win | 42.789878 |
| **Andrew Jackson** | 1828 | Democratic | 642806 | win | 56.203927 |
| **John Quincy Adams** | 1828 | National Republican | 500897 | loss | 43.796073 |
| **Andrew Jackson** | 1832 | Democratic | 702735 | win | 54.574789 |

# Les indices ne sont pas nécessairement uniques

Les indices ne sont pas nécessairement uniques.

- À gauche : les valeurs de l'*index* sont uniques et numériques, et jouent le rôle de numéros de ligne (cas par défaut).
- À droite : les valeurs de l'*index* sont nommées et non uniques.

| | Candidate | Party | % | Year | Result |
|---|---|---|---|---|---|
| 0 | Obama | Democratic | 52.9 | 2008 | win |
| 1 | McCain | Republican | 45.7 | 2008 | loss |
| 2 | Obama | Democratic | 51.1 | 2012 | win |
| 3 | Romney | Republican | 47.2 | 2012 | loss |
| 4 | Clinton | Democratic | 48.2 | 2016 | loss |
| 5 | Trump | Republican | 46.1 | 2016 | win |

| Year | Candidate | Party | % | Result |
|---|---|---|---|---|
| 2008 | Obama | Democratic | 52.9 | win |
| 2008 | McCain | Republican | 45.7 | loss |
| 2012 | Obama | Democratic | 51.1 | win |
| 2012 | Romney | Republican | 47.2 | loss |
| 2016 | Clinton | Democratic | 48.2 | loss |
| 2016 | Trump | Republican | 46.1 | win |

# Modification d'Indices

- On peut sélectionner une nouvelle colonne et la définir comme index du DataFrame.

```
elections = elections.set_index("Candidate")
```

| Candidate | Year | Party | Popular vote | Result | % |
|---|---|---|---|---|---|
| Andrew Jackson | 1824 | Democratic-Republican | 151271 | loss | 57.210122 |
| John Quincy Adams | 1824 | Democratic-Republican | 113142 | win | 42.789878 |
| Andrew Jackson | 1828 | Democratic | 642806 | win | 56.203927 |
| John Quincy Adams | 1828 | National Republican | 500897 | loss | 43.796073 |
| Andrew Jackson | 1832 | Democratic | 702735 | win | 54.574789 |
| ... | ... | ... | ... | ... | ... |
| Donald Trump | 2024 | Republican | 77303568 | win | 49.808629 |
| Kamala Harris | 2024 | Democratic | 75019230 | loss | 48.336772 |
| Jill Stein | 2024 | Green | 861155 | loss | 0.554864 |
| Robert Kennedy | 2024 | Independent | 756383 | loss | 0.487357 |
| Chase Oliver | 2024 | Libertarian Party | 650130 | loss | 0.418895 |

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| 0 | 1824 | Andrew Jackson | Democratic-Republican | 151271 | loss | 57.210122 |
| 1 | 1824 | John Quincy Adams | Democratic-Republican | 113142 | win | 42.789878 |
| 2 | 1828 | Andrew Jackson | Democratic | 642806 | win | 56.203927 |
| 3 | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| 4 | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |
| ... | ... | ... | ... | ... | ... | ... |
| 182 | 2024 | Donald Trump | Republican | 77303568 | win | 49.808629 |
| 183 | 2024 | Kamala Harris | Democratic | 75019230 | loss | 48.336772 |
| 184 | 2024 | Jill Stein | Green | 861155 | loss | 0.554864 |
| 185 | 2024 | Robert Kennedy | Independent | 756383 | loss | 0.487357 |
| 186 | 2024 | Chase Oliver | Libertarian Party | 650130 | loss | 0.418895 |



# Réinitialisation de *l'Indice (index)*

- Réinitialiser *l'indice (index)* à la liste d'entiers par défaut.

```
elections = elections.reset_index()
```

| Candidate | Year | Party | Popular vote | Result | % |
|---|---|---|---|---|---|
| Andrew Jackson | 1824 | Democratic-Republican | 151271 | loss | 57.210122 |
| John Quincy Adams | 1824 | Democratic-Republican | 113142 | win | 42.789878 |
| Andrew Jackson | 1828 | Democratic | 642806 | win | 56.203927 |
| John Quincy Adams | 1828 | National Republican | 500897 | loss | 43.796073 |
| Andrew Jackson | 1832 | Democratic | 702735 | win | 54.574789 |
| ... | ... | ... | ... | ... | ... |
| Donald Trump | 2024 | Republican | 77303568 | win | 49.808629 |
| Kamala Harris | 2024 | Democratic | 75019230 | loss | 48.336772 |
| Jill Stein | 2024 | Green | 861155 | loss | 0.554864 |
| Robert Kennedy | 2024 | Independent | 756383 | loss | 0.487357 |
| Chase Oliver | 2024 | Libertarian Party | 650130 | loss | 0.418895 |

| | Year | Candidate | Party | Popular vote | Result | % |
|---|---|---|---|---|---|---|
| 0 | 1824 | Andrew Jackson | Democratic-Republican | 151271 | loss | 57.210122 |
| 1 | 1824 | John Quincy Adams | Democratic-Republican | 113142 | win | 42.789878 |
| 2 | 1828 | Andrew Jackson | Democratic | 642806 | win | 56.203927 |
| 3 | 1828 | John Quincy Adams | National Republican | 500897 | loss | 43.796073 |
| 4 | 1832 | Andrew Jackson | Democratic | 702735 | win | 54.574789 |
| ... | ... | ... | ... | ... | ... | ... |
| 182 | 2024 | Donald Trump | Republican | 77303568 | win | 49.808629 |
| 183 | 2024 | Kamala Harris | Democratic | 75019230 | loss | 48.336772 |
| 184 | 2024 | Jill Stein | Green | 861155 | loss | 0.554864 |
| 185 | 2024 | Robert Kennedy | Independent | 756383 | loss | 0.487357 |
| 186 | 2024 | Chase Oliver | Libertarian Party | 650130 | loss | 0.418895 |



## Pourquoi utiliser [] ?

- En résumé : [ ] peut être beaucoup plus concis que .loc ou .iloc.

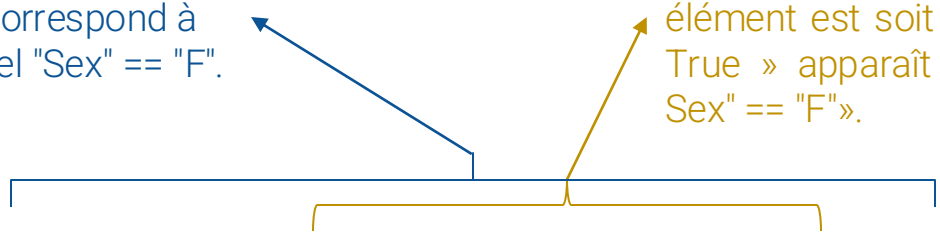
- Exemple : pour extraire la colonne « Candidate », il est bien plus simple d'écrire `elections["Candidate"]` que `elections.loc[:, "Candidate"].`

# Les tableaux booléens

On peut utiliser **`.loc`**.

`DataFrame` de longueur 239 537 dont chaque entrée correspond à un prénom pour lequel "Sex" == "F".

`Series` de longueur 407 428 dont chaque élément est soit « True » soit « False », où « True » apparaît pour chaque prénom dont Sex" == "F"».



```
babynames.loc[babynames["Sex"] == "F", :]
```

| | State | Sex | Year | Name | Count |
|---|---|---|---|---|---|
| **0** | CA | F | 1910 | Mary | 295 |
| **1** | CA | F | 1910 | Helen | 239 |
| **2** | CA | F | 1910 | Dorothy | 220 |
| **3** | CA | F | 1910 | Margaret | 163 |
| **4** | CA | F | 1910 | Frances | 134 |
| **...** | ... | ... | ... | ... | ... |
| **239532** | CA | F | 2022 | Zemira | 5 |
| **239533** | CA | F | 2022 | Ziggy | 5 |
| **239534** | CA | F | 2022 | Zimal | 5 |
| **239535** | CA | F | 2022 | Zosia | 5 |
| **239536** | CA | F | 2022 | Zulay | 5 |

239537 rows × 5 columns

# Les tableaux booléens

Les tableaux booléens peuvent être générés en utilisant des opérateurs logiques sur `Series`.

`Series` de longueur 407 428 dont chaque élément est soit « True » soit « False », où « True » apparaît pour chaque prénom dont Sex" == "F"».

```
babynames["Sex"]
```

```
0         F
1         F
2         F
3         F
4         F
         ..
407423    M
407424    M
407425    M
407426    M
407427    M
Name: Sex, Length: 407428, dtype: object
```

```
(babynames["Sex"] == "F")
```

```
0          True
1          True
2          True
3          True
4          True
          ...
407423    False
407424    False
407425    False
407426    False
407427    False
Name: Sex, Length: 407428, dtype: bool
```

# Les tableaux booléens

Nous pouvons utiliser notre nouveau tableau booléen pour filtrer les lignes souhaitées.
Masque booléen !

`DataFrame` de longueur 239 537 dont chaque entrée correspond à un prénom pour lequel "Sex" == "F".

`Series` de longueur 407 428 dont chaque élément est soit « True » soit « False », où « True » apparaît pour chaque prénom dont Sex" == "F"».

```
babynames[(babynames["Sex"] == "F")]
```

| | State | Sex | Year | Name | Count |
|---|---|---|---|---|---|
| **0** | CA | F | 1910 | Mary | 295 |
| **1** | CA | F | 1910 | Helen | 239 |
| **2** | CA | F | 1910 | Dorothy | 220 |
| **3** | CA | F | 1910 | Margaret | 163 |
| **4** | CA | F | 1910 | Frances | 134 |
| **...** | ... | ... | ... | ... | ... |
| **239532** | CA | F | 2022 | Zemira | 5 |
| **239533** | CA | F | 2022 | Ziggy | 5 |
| **239534** | CA | F | 2022 | Zimal | 5 |
| **239535** | CA | F | 2022 | Zosia | 5 |
| **239536** | CA | F | 2022 | Zulay | 5 |

239537 rows × 5 columns



# Sélection booléenne : alternatives plus concises

La sélection à l'aide de tableaux booléens est utile, mais elle peut rendre le code trop verbeux.

```
babynames[(babynames["Name"] == "Bella") |
          (babynames["Name"] == "Alex") |
          (babynames["Name"] == "Narges") |
          (babynames["Name"] == "Lisa")]
```

| | | | | | |
|---|---|---|---|---|---|
| **6289** | CA | F | 1923 | Bella | 5 |
| **7512** | CA | F | 1925 | Bella | 8 |
| **12368** | CA | F | 1932 | Lisa | 5 |
| **14741** | CA | F | 1936 | Lisa | 8 |
| **17084** | CA | F | 1939 | Lisa | 5 |
| **...** | ... | ... | ... | ... | ... |
| **393248** | CA | M | 2018 | Alex | 495 |
| **396111** | CA | M | 2019 | Alex | 438 |
| **398983** | CA | M | 2020 | Alex | 379 |
| **401788** | CA | M | 2021 | Alex | 333 |
| **404663** | CA | M | 2022 | Alex | 344 |

317 rows × 5 columns

`pandas` propose **des** alternatives, plus concise, par exemple:

- `.isin`
- `.str.startswith`
- `.groupby.filter (à revoir)`

# Sélection booléenne : alternatives plus concises

Pour simplifier le filtrage booléen, `pandas` propose **plusieurs** fonctions, par exemple:

- `.isin`
- `.str.startswith`
- `.groupby.filter`

```
names = ["Bella", "Alex", "Narges", "Lisa"]
babynames[babynames["Name"].isin(names)]
```

Renvoie une `Series` booléenne dont les valeurs sont `True` lorsque le prénom correspondant dans `babynames` Bella, Alex, Narges, ou Lisa.

```
0         False
1         False
2         False
3         False
4         False
          ...
407423    False
407424    False
407425    False
407426    False
407427    False
Name: Name, Length: 407428, dtype: bool
```

| | | | | | |
|---|---|---|---|---|---|
| **6289** | CA | F | 1923 | Bella | 5 |
| **7512** | CA | F | 1925 | Bella | 8 |
| **12368** | CA | F | 1932 | Lisa | 5 |
| **14741** | CA | F | 1936 | Lisa | 8 |
| **17084** | CA | F | 1939 | Lisa | 5 |
| ... | ... | ... | ... | ... | ... |
| **393248** | CA | M | 2018 | Alex | 495 |
| **396111** | CA | M | 2019 | Alex | 438 |
| **398983** | CA | M | 2020 | Alex | 379 |
| **401788** | CA | M | 2021 | Alex | 333 |
| **404663** | CA | M | 2022 | Alex | 344 |

317 rows × 5 columns